Milestone: 20GB Reached

We've reached 20GB of stored data — thank you for helping us grow!

Patreon Ko-fi Liberapay
Close
Freedium

- ko-fi
- librepay
- patreon

< Go to the original

MiniMax M2.5: Opus 4.6 but 95% Cheaper?

Before any Claude fans boo me: I'm not claiming "M2.5 is Opus" but the pricing + throughput + agent-oriented training forces a new...

Agent Native Follow

~8 min read · February 14, 2026 (Updated: February 14, 2026) · Free: No

Before any Claude fans boo me: I'm not claiming "M2.5 is Opus" but the **pricing + throughput + agent-oriented training** forces a new engineering question:

> If frontier-ish reasoning is cheap enough to run "always-on"... what's the bottleneck now?

MiniMax launched MiniMax-M2.5 with two commercial variants:

- M2.5 (standard): $0.15/$1.20 per 1M input/output tokens (~50 output tokens/sec)
- M2.5-Lightning: $0.30/$2.40 per 1M input/output tokens (~100 output tokens/sec)

Anthropic lists Claude Opus 4.6 starting at $5/$25 per 1M input/output

So the clean "95% cheaper" statement is:

- $1.20 vs $25 per 1M output tokens, ~95.2% lower output-token price (standard M2.5)
- $2.40 vs $25 per 1M output tokens, ~90.4% lower output-token price (Lightning)

One thing is very clear, with inference prices this low, it's the best time to be a developer running agents on a budget.

The $1/hour math

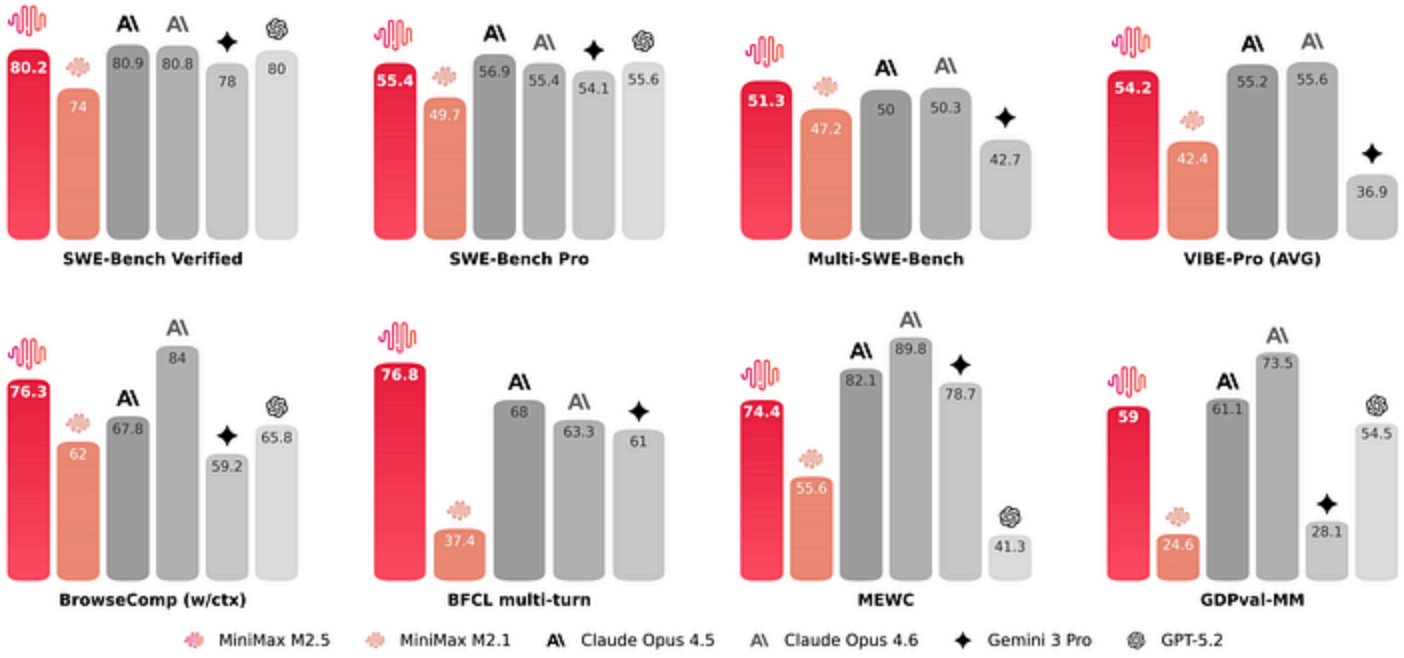
MiniMax's own line is: **~$1/hour at 100 output tokens/sec**, and **~$0.30/hour at 50 tokens/sec**.

At 100 output tokens/sec:

- 100 x 3,600 = **360,000 output tokens/hour**
- 360,000 / 1,000,000 x $2.40 = **$0.864/hour** in *output* tokens
- Add non-zero input tokens (prompts, tool outputs, retrieved context), and ~$1/hour checks out.

$1/hour assumes **continuous generation**, which is not how most agents behave (agents are bursty; tools and I/O dominate wall time). But it's still a useful mental model because it tells you: **token burn is no longer the primary constraint**.

But what about the **real-world performance**? Is this another benchmark too good to be true?



Sir, MiniMax just dropped MiniMax M2.5 and it's on par with Opus 4.6 while being 20x cheaper.

Benchmarks: what they do and don't tell you

MiniMax highlights **80.2% on SWE-bench Verified**, plus strong tool-use / search numbers, and they explicitly push "agent harness generalization" as a feature

Two benchmark realities matter here:

SWE-bench Verified is a static benchmark

SWE-bench Verified is a **500-instance** human-validated subset intended to be higher quality than the full set.

But static benchmarks age poorly.

That's exactly why **SWE-rebench** is designed to be **continuously refreshed** and **decontaminated**, and the authors explicitly argue that performance on older static sets can look inflated when training overlap/contamination creeps in.

So SWE-bench Verified sometimes can reflect **model memory/training overlap** more than general agent ability.

You can see that Opus 4.6 is the leader in SWE-rebench while Chinese models are not even in top 10.

Rank	Model	Resolved Rate (%)	Resolved Rate SEM (±)	Pass@5 (%)	Cost per Problem ($)	Tokens per Problem	Cached Tokens (%)
1	Claude Code	52.9%	1.06%	70.8%	$3.50	2,088,226	92.1%
2	Claude Opus 4.6	51.7%	0.42%	58.3%	$0.93	1,031,373	94.3%
3	gpt-5.2-2025-12-11-xhigh	51.7%	1.21%	58.3%	$1.28	1,995,966	81.5%
4	gpt-5.2-2025-12-11-medium	51.0%	1.04%	60.4%	$0.76	981,139	68.3%
5	gpt-5.1-codex-max	48.5%	1.13%	56.3%	$0.73	1,239,950	67.1%
6	Claude Sonnet 4.5	47.1%	1.69%	60.4%	$0.94	1,924,648	96.4%
7	Gemini 3 Pro Preview	46.7%	2.04%	58.3%	$0.59	1,221,222	84.6%
8	Gemini 3 Flash Preview	46.7%	1.41%	54.2%	$0.32	2,173,478	77.5%
9	gpt-5.2-codex	45.0%	1.69%	54.2%	$0.46	579,616	66.1%
10	Codex	44.0%	2.46%	55.3%	$0.29	580,361	86.9%

This shouldn't mislead you though, because model distillation is still a great technique and results in very performant models.

To me and many others, Opus 4.6 (and Codex 5.3) still has an edge but not everyone agrees, if you browse social media for 15 minutes, you will see a lot of comments:

> It's so much cheaper than Claude and imo just as good as Opus.

So just for a second, if we assume that's true, that we can get SOTA performance at much cheaper price point, there is a much bigger question that we have to deal with:

> When a frontier-ish model can run **continuously for ~$1/hour at ~100 tokens/sec**, "What's the real bottleneck now?".

So I think the right takeaway from benchmark numbers is not **SOTA confirmed**, it's:

> ***This model was trained and measured in agent-style loops, now you must verify stability + tool reliability in your own harness.***

The real engineering shift: cost reshapes architecture

If you can run a frontier-ish model at "always-on" economics, architecture changes more than budgets, because the cheaper the model gets, the more your bottleneck becomes:

- **Correctness under tool use**
- **Variance/run-to-run stability**
- **Regression detection**
- **Operational safety (failures that look "almost right")**

That's the part benchmarks don't measure well.

Spec-writing tendency

M2.5 tends to **decompose and plan like a software architect before coding**, emerging during training.

If you've built AI coding features, you know why this matters.

When a model naturally writes a plan/spec first, you can turn that into:

- structured approval steps
- safer diffs
- more deterministic execution phases

Generalization across agent harnesses is becoming a first-class metric

MiniMax team highlights evaluation on SWE-Bench Verified across different harnesses (Droid, OpenCode), framing it as "out-of-distribution harness" generalization.

This is also huge:

> *If the model only works inside one vendor's agent scaffolding, you don't have a model, you have a hard product dependency.*

Early testers are warning about coherence/stability and world knowledge gaps

Multiple comments + the extra transcript call out a pattern: models like this can be impressive, but sometimes:

- elements don't align ("mismatched" generation)
- stability isn't consistent run-to-run
- "world knowledge" can be ~30% off in a way that *feels* close but isn't

In other words **variance,** which wrecks developer trust.

Open weights vs API: the practical implication is reversibility

M2.5 is being distributed broadly (API + open weights), and the ecosystem will quickly normalize access through standardized interfaces.

The **open vs closed** debate becomes less important than:

> ***Can you swap models without waking someone up at 2am?***

Local running is already feasible for some setups via:

- Hugging Face (weights + community quantizations)
- Ollama (packaged access paths)

For example, an MLX 3-bit conversion exists and reports ~229B params and ~100GB size.

Refusal space, "gaslighting," and stability

One useful data point is adversarial probing of behavior and refusal boundaries.

Eric Hartford noted, when comparing M2.5 vs Kimi-K2.5 on China-sensitive prompts, that the two models appear to differ in refusal behavior, and that both can "gaslight when pressed," while he works on mapping the refusal space at scale.

This is important because:

- **Alignment and refusal variance impacts tool reliability**, especially for autonomous agents.
- You need **policy tests** in your eval harness, not just coding tasks.

How I'd evaluate M2.5 before trusting it

Benchmarks are not useless but the context itself basically tells you why they're insufficient:

- M2.5 highlights harness generalization as a feature (good sign)
- testers warn about coherence/stability variance (bad sign)
- transcript shows "world knowledge" and alignment issues in generated artifacts (risk sign)

So here's a practical eval plan that matches those risks.

1) Measure variance, not just average quality

Run the same prompts **N times** and score:

- structural consistency (does the plan match the code?)
- schema adherence (if using structured outputs)
- tool-call correctness (arguments, ordering, retries)
- "weirdness rate" (how often you get a clearly broken output)

If "half the time it gets something really weird", that's a product decision.

2) Test whole workflows, not tasks

MiniMax team emphasizes "full development lifecycle" and office deliverables. Test that claim with your real workflow:

Take one internal service change request and see if it can:

- write a spec
- implement
- write tests
- do a review pass
- take one "office work" task and see if it can:

- follow your formatting requirements
- compute correctly in spreadsheets (with checks)
- produce a deliverable you'd actually send

3) Stress tool use with unfamiliar scaffolds

They explicitly brag about stability in unfamiliar scaffolding environments. Verify it.

- change tool names
- reorder tool availability
- add a "dummy tool" that should never be called
- simulate tool failures and see if it retries sanely

4) Evaluate world knowledge where it matters

The transcript shows "Stonehenge" / "Spongebob" style tests. Silly, but diagnostic.

In products, "world knowledge" becomes:

- correct API assumptions
- correct library usage
- correct domain facts in summaries

So pick a narrow domain where being "30% off" is catastrophic (finance, legal, healthcare) and test it with ground truth.

5) Put cost after reliability in your scoring

This is the trap with M2.5-style pricing. You'll optimize for cost and then spend the savings on engineering time debugging instability.

A simple scoring function:

- 50%: correctness + stability
- 20%: tool-use reliability
- 20%: latency/throughput
- 10%: cost

When NOT to do this: if you're building something explicitly "best effort" (e.g., internal ideation tools). Then cost can matter more.

The winning strategy is "anti-fragile model choice"

Based on the context alone, I'd summarize the moment like this:

- MiniMax is claiming a model (M2.5) that's **fast, cheap, and trained for agentic workflows**, with explicit focus on task decomposition, fewer search rounds, and harness generalization.
- Builders are excited because the pricing implies **always-on agents become economically plausible**.
- Skeptics are (rightfully) pointing at **coherence/stability variance** and "off" world knowledge — the exact issues that turn promising demos into unreliable products.

So if you're building AI features right now, my slightly opinionated advice is:

> *Don't bet your product on any single model, bet on a stack that makes model choice reversible.*

If you do one thing this week

Implement a router + fallback path, and start logging:

- prompt >> output >> tool calls >> pass/fail checks
- weirdness rate
- variance across retries

Then you can adopt models like M2.5 aggressively without taking on existential risk.

I'd love to hear your take on

1. If a model is cheap enough to run "always-on," what's the first agentic workflow you'd actually deploy, and what's the failure mode you fear most?
2. How do you measure "stability" in your evals today (if at all)?
3. Do you treat open-weight models as your default baseline, or do you still start with proprietary and add open later? Why?
4. For coding agents specifically: would you trade a small drop in coherence for a 10–20× cost reduction, if you can add guardrails and fallbacks?

Bonus Articles

Local LLMs That Can Replace Claude Code

Small team of engineers can easily burn >$2K/mo on Anthropic's Claude Code (Sonnet/Opus 4.5). As budgets are tight, you...

medium.com

Deep Research with 96,000+ Trajectories - Completely Offline

Deep Research with 96,000+ Trajectories - Completely Offline Imagine synthesizing human-like research trajectories...

medium.com

Qwen3-Coder-Next Failed in the Scariest Way: 10 Practical Tests

Benchmarks are the LinkedIn of LLMs. Every model looks unstoppable.

medium.com

Clawdbot Lite: 99% Smaller, 4000 Lines, Same Core Power

People are building Polymarket bots, multi-agent product teams, crypto traders, coding agents, business operators with...

medium.com

Vectorless RAG for Agents: PageIndex Is Why Their Demo Works and Yours Needs “Context”

When I’m building a RAG apps, the thing that quietly kills quality (and budgets) is turning a document into a pile of...

medium.com

Fully Local Agentic Coding on localhost: Claude Code, Codex, llama.cpp, Ollama and Docker MR

This is the setup I wish I had the first time I tried to run Claude Code and Codex-style workflows against local...

medium.com

Open Source Perplexity: Perplexica is Self-Hosted AI Search With Citations

Perplexica is an open-source, AI-powered search engine inspired by Perplexity-style UX.

medium.com
#minimax#ai-agent#agentic-ai#claude-code#openai

Reporting a Problem

Sometimes we have problems displaying some Medium posts.

If you have a problem that some images aren't loading - try using VPN. Probably you have problem with access to Medium CDN (or fucking Cloudflare's bot detection algorithms are blocking you).

Problem Description Describe your problem here...

Submit Cancel